

SaaS & E-Commerce Customer Churn & Revenue Intelligence

Comprehensive Resume, Data Provenance & Technical Interview Preparation Master Guide

1. Data Provenance & Origin (Data Kahan Se Aaya & Kiska Hai?)

Q: Is this real customer data, or where did it come from?

This dataset is an **Enterprise-Simulated SaaS & E-Commerce Telemetry Dataset (12,500 Customer Accounts, \$19.02M ARR)** modeled strictly on real-world subscription unit economics from benchmark B2B/B2C SaaS companies (like Stripe, Chargebee, HubSpot, and Salesforce).

How to answer in an interview:

'The data represents a mid-market multi-tier B2B SaaS and subscription commerce enterprise operating across Starter, Professional, Enterprise, and Growth tiers. Because enterprise customer transaction telemetry contains sensitive PII and confidential financial records, the dataset was generated using rigorous probabilistic business rules—reflecting authentic distributions of churn curves, ticket escalation frequencies, payment gateway failure patterns, and customer health score decay.'

Key Parameters & Realism Built into the Data:

- **12,500 Customer Profiles:** Spanning 4 plans (\$49/mo Starter to \$999/mo Enterprise) across 2 years of history.
- **Multi-Table Relational Schema:** Telemetry spans 4 raw sources: Customers, Subscriptions, Usage Events, and Support Tickets.
- **True Business Signals:** Involuntary churn (payment failure), voluntary churn (usage drop >40%), ticket resolution delays, and contract duration effects.

2. Resume Description & STAR Bullet Points

Project Title: SaaS Customer Churn & Revenue Intelligence Platform

Core Stack: SQL, Python (Pandas, Scikit-Learn), Power BI (DAX), SQLite, Star Schema Data Modeling

- **Data Engineering & Architecture:** Built an end-to-end analytics pipeline transforming 12,500 customer transaction records (\$19.02M ARR) into an optimized Star Schema SQLite warehouse (Fact_Subscriptions, Dim_Customers, Dim_Plans, Dim_Date).
- **Advanced SQL Cohort Modeling:** Authored complex SQL CTEs and Window Functions (LAG, LEAD, ROW_NUMBER) to calculate dynamic Month-0 to Month-12 cohort retention matrices and subscription expansion velocity.
- **Predictive Machine Learning:** Trained and deployed a Logistic Regression churn risk classifier achieving **87.5% Recall** (0.88 ROC-AUC), identifying high-risk accounts 30-60 days before contract expiration.
- **Executive Power BI Dashboard:** Engineered a 3-page interactive Power BI reporting suite backed by **26 production DAX measures** (MRR, ARR, NRR, LTV, ARPU, Quick Ratio) with dynamic drill-throughs.
- **Financial Strategy & ROI:** Modeled 4 proactive retention interventions projected to recover **\$1.18M in at-risk ARR** at an estimated **737.5% first-year ROI**.

3. Skills & Technologies Used

Category	Technology / Tool	Key Applied Use Case in Project
Languages	Python 3.10+, SQL	ETL pipelines, data cleaning, ML model training, and CTE analytics
BI & Visualization	Power BI Desktop, DAX	3-page executive suite, 26 DAX measures, Star schema data model
Database & Storage	SQLite, Relational Star Schema	Fact & Dimension tables, primary/foreign key indexing, ACID queries
Data Science & ML	Pandas, Scikit-Learn, NumPy	Feature engineering, logistic regression, confusion matrix, ROC-AUC
Version Control	Git, GitHub	Source code versioning, automated workflows, technical markdown specs

4. End-to-End Workflow: Step-by-Step Breakdown

- Step 1: Data Ingestion & ETL:** Cleaned raw transaction data, handled null values, normalized date timestamps, and cast data types using automated Python Pandas scripts.
- Step 2: Star Schema Modeling:** Designed 1 central Fact table (Fact_Subscriptions) and 4 Dimension tables (Dim_Customers, Dim_Plans, Dim_Date, Dim_ChurnReason) in SQLite to optimize DAX analytical queries.
- Step 3: SQL Cohort Retention Analytics:** Wrote multi-stage CTEs calculating Month 0 to Month 12 retention rates, identifying that Month 3 experienced the steepest retention drop (18.4% drop).
- Step 4: Machine Learning Risk Scoring:** Engineered features (login decay, unresolved tickets, billing retry count) and trained a classification model with 87.5% Recall to flag churners before expiration.
- Step 5: Executive Power BI Reporting:** Created 3 dashboards: Executive SaaS Overview, Cohort Lifecycle, and Risk Intelligence with 26 DAX measures (MRR, NRR, LTV, Quick Ratio).
- Step 6: Financial Impact Modeling:** Calculated \$1.18M ARR recoverable across 4 targeted interventions with an implementation cost of \$141k (737.5% ROI).

5. 60-Second Interview Elevator Pitch

'In my recent project, I built an end-to-end SaaS Churn and Revenue Intelligence platform analyzing 12,500 customer accounts and \$19.02M in ARR. The primary business problem was high voluntary churn in the early 90 days and involuntary churn due to billing failures.

I designed a Star Schema SQLite data warehouse via an automated Python ETL pipeline, wrote advanced SQL CTEs and Window Functions for M0–M12 cohort retention matrices, and trained an ML classification model with 87.5% recall to predict churn ahead of time.

Finally, I built a 3-page Power BI dashboard with 26 DAX measures and modeled 4 strategic business interventions that can recover \$1.18M in ARR at a 737.5% ROI.'

6. Top 10 Technical & Business Interview Questions & Answers

Q1: Why did you choose a Star Schema over a single flat CSV table?

Flat tables create massive data redundancy and slow down analytical queries. A Star Schema separates numerical measurements (Fact_Subscriptions) from descriptive attributes (Dim_Customers, Dim_Plans, Dim_Date). This enables Power BI's VertiPaq engine to perform highly efficient 1-to-many relationship filtering, reduces memory footprint, and enforces referential integrity.

Q2: How did you calculate Cohort Retention in SQL? Walk me through the query logic.

I structured a 3-stage SQL CTE: (1) First CTE finds each customer's MIN(start_date) to assign their Cohort Month. (2) Second CTE joins all subsequent active subscription months, calculating Month_Offset (0 to 12) via DATEDIFF. (3) Final query calculates COUNT(DISTINCT customer_id) active in each offset divided by the base Month 0 cohort size to derive the retention percentage.

Q3: Why did you prioritize Recall over Accuracy or Precision in your ML Churn model?

In customer churn prediction, False Negatives (predicting a churning customer as 'Safe') are extremely expensive because the business loses the chance to intervene and retain them. False Positives only result in an extra customer success outreach email. Optimizing for Recall (87.5%) ensures we capture the maximum volume of at-risk revenue.

Q4: What is Net Revenue Retention (NRR) and how did you write its DAX formula?

NRR measures the percentage of recurring revenue retained from existing customers over a period, factoring in expansions (upgrades) minus contractions (downgrades) and churn. An NRR > 100% indicates organic account expansion. DAX Formula: $[NRR \ %] = \text{DIVIDE}([Starting \ MRR] + [Expansion \ MRR] - [Churn \ MRR] - [Downgrade \ MRR], [Starting \ MRR], 0)$

Q5: What were the top 3 root causes of churn identified in your analysis?

1. **90-Day Onboarding Cliff:** Highest drop occurred between Month 1 and Month 3 on monthly plans.
2. **Support Ticket Escalation:** Accounts with ≥ 3 unresolved high-priority tickets had a 4.2x higher churn hazard rate.
3. **Payment Gateway Delays (Involuntary Churn):** 22% of churn was due to expired cards and repeated billing retries.

Q6: How did you handle data cleaning and missing values in Python ETL?

Missing cancellation dates were explicitly kept as NULL to represent active subscriptions without corrupting tenure calculations. Plan names and categorical dimensions were standardized (trimmed whitespace, uppercase normalization) to prevent join mismatches. Primary and Foreign key constraints were validated before loading into SQLite.

Q7: How did you calculate the \$1.18M ARR recovery and 737.5% ROI figure?

Total at-risk ARR was ~\$3.5M. I modeled 4 interventions with conservative recovery rates: (1) Smart Dunning/Payment Retries (\$280k saved), (2) Customer Success Outreach (\$420k saved), (3) 90-Day Onboarding Overhaul (\$350k saved), and (4) Annual Plan Discount Incentives (\$130k saved). Total saved = \$1.18M. Subtracting the \$141k program implementation cost yielded an ROI of $(\$1.18M - \$141k)/\$141k = 737.5\%$.

Q8: What is the SaaS Quick Ratio and why is it important?

The Quick Ratio evaluates revenue growth reliability: $([New \ MRR] + [Expansion \ MRR]) / ([Churn \ MRR] + [Contraction \ MRR])$. A ratio > 4.0 indicates hyper-growth efficiency, while < 1.0 indicates a shrinking business. In our dataset, it stood at 2.85.

Q9: How did you design the Customer Health Score?

It is a weighted 0–100 composite index: 35% Product Usage Frequency (login trends), 25% Feature Adoption Breadth, 20% Support Ticket Sentiment/Resolution, and 20% Payment Reliability. Scores <40 are classified as 'Critical Risk', 40–70 as 'Moderate Risk', and >70 as 'Healthy'.

Q10: If data scaled to 100M+ event rows, how would you modify this architecture?

I would migrate SQLite to a cloud data warehouse like Snowflake or Google BigQuery. The Python ETL script would transition to Apache Spark / PySpark or dbt for scalable transformation, and Power BI would switch to Composite Mode with Aggregation Tables over DirectQuery.